

Table des matières

Exercices 03 — dbt et la couche STAGING	1
Questions courtes	1
Questions pratiques	2
Diagnostics	3
Questions à développer	3
Mises en situation	4

Exercices 03 — dbt et la couche STAGING

Comment travailler ces exercices. Répondez seul, puis vérifiez avec le corrigé. Les exercices marqués [labo] portent sur ce que vous venez de faire au labo ; les autres vérifient que le cours est clair dans votre tête. Tout vient de la Théorie 03 et du Labo 03.

Le corrigé est dans `Corrige_03.md`. Répondez à l'énoncé en entier avant de l'ouvrir : c'est en butant sur une question qu'on découvre ce qu'on avait lu sans le comprendre.

Questions courtes

1. Un modèle. Dites ce que contient un fichier de modèle dbt, et ce qu'il ne contient jamais.
2. Ce que dbt ajoute. Nommez les quatre choses que dbt apporte autour d'un `SELECT`, et dites laquelle relève du moteur de base de données.
3. Le nom de l'objet. D'où vient le nom de la table ou de la vue que dbt crée ?
4. Les deux fichiers. Distinguez `dbt_project.yml` et `profiles.yml` : ce que chacun déclare, et lequel se versionne.
5. Pourquoi ailleurs. Sous DuckDB, `profiles.yml` ne porte qu'un chemin de fichier. Donnez la raison pour laquelle il reste quand même hors du dépôt.
6. La cascade. Expliquez comment les réglages du bloc `models` : s'appliquent, et ce que cela évite d'écrire.
7. Une source. Définissez ce qu'est une source pour dbt, et l'engagement qu'on prend en la déclarant.
8. Quatre gains. Écrire `from raw.erp_kna1` fonctionne. Nommez ce que `source()` apporte de plus, et celui des gains qui se mesure le plus vite.
9. Le schéma qui s'ajoute. Un profil vise `staging`, un dossier déclare `+schema: marts`. Où dbt écrit-il, et pourquoi ce comportement existe-t-il ?

10. La macro. Dites ce que fait `generate_schema_name` dans la version de ce cours, en deux branches.
 11. Les quatre gestes. Nommez les quatre gestes du `STAGING`, avec un exemple de chacun chez Boréal.
 12. Le geste refusé. Quel geste le `STAGING` refuse-t-il, et quelle est la raison de fond ?
 13. Le geste oublié. Des quatre gestes, lequel oublie-t-on, et que se passe-t-il si on l'oublie ?
 14. L'état courant. Expliquez comment on tire l'état courant d'une table livrée par lots, en nommant la fonction employée.
 15. Ce que l'état courant répare. Chez Boréal, un contrôle des commandes orphelines rend 0. Dites pourquoi ce zéro est le produit d'un choix.
 16. Quatre matérialisations. Nommez-les, avec ce que `dbt` écrit dans chaque cas.
 17. Le staging en vues. Donnez les trois raisons qui justifient de matérialiser le staging en vues.
 18. `ref()`. Dites ce que cette fonction remplace, et les trois choses que `dbt` en tire.
 19. Nommer en dur. Un modèle qui écrit `from staging.stg_commandes` au lieu de `ref()` fonctionne. Dites ce qui casse.
 20. Trois mots proches. Distinguez, sur Snowflake, une table transient, le schéma `STAGING` d'un projet `dbt`, et un stage.
 21. [labo] Le fichier vide. Que se passe-t-il si `profiles.yml` existe mais est vide, et en quoi est-ce pire qu'un fichier absent ?
 22. [labo] Les deux comptes. `stg_commandes` rend 25 483 lignes là où sa source en porte 26 538. Expliquez l'écart.
-

Questions pratiques

- P1. [labo] Une vue de plus. Écrivez `stg_articles.sql`, la vue de staging des articles, à partir de `raw.erp_mara`. Elle doit tirer l'état courant, décoder la gamme et le type d'article d'après le dictionnaire, et traiter l'année-modèle vide comme une absence.
- P2. Vérifier la vue. Écrivez la requête qui compte les lignes de la source, celles de la vue, et les gammes non décodées. Donnez les trois résultats attendus.
- P3. Déclarer sa source. Donnez le YAML à ajouter dans `_sources.yml` pour que la question P1 soit possible, et dites ce qui se passe si vous l'oubliez.
- P4. Voir le SQL compilé. Donnez la commande qui produit le SQL réellement envoyé à DuckDB pour un modèle, dites où le fichier atterrit, et ce que `source()` y est devenu.
- P5. Lire l'aval. Donnez la commande qui liste `stg_commandes` et tout ce qui en dépend. Donnez le résultat attendu sur le projet du labo.
- P6. Lire l'amont. Donnez la commande qui liste tout ce dont `ctl_commandes_sans_concessionnaire` dépend, sources comprises, et le résultat attendu.

P7. Changer une matérialisation. Donnez les deux façons de matérialiser `stg_articles` en table plutôt qu'en vue, et la requête qui vérifie que c'en est bien une.

P8. Le contrôle du décodage. Écrivez la requête qui, sur `stg_articles`, vérifie d'un coup que le décodage de la gamme et celui du type d'article ont fonctionné.

Diagnostics

E1. Le graphe invisible. Un collègue écrit ce modèle de contrôle :

```
select count(*) as commandes
from staging.stg_commandes
```

Il tourne, `dbt run` rend `PASS=1`, et le résultat est juste. Dites ce qui ne va pas, comment le constater en une commande, et les deux conséquences concrètes.

E2. Le staging naïf. Un collègue écrit sa vue de staging des commandes ainsi :

```
select
  vbeln as numero_commande,
  kunnr as code_concessionnaire,
  netwr:decimal(18, 2) as valeur_nette
from {{ source('raw', 'erp_vbak') }}
```

Elle tourne, et le total du chiffre d'affaires part en production. Dites ce qui manque, de combien le total est faux, et pourquoi personne ne le verra.

E3. Le schéma doublé. Un collègue monte son projet, lance `dbt run`, et trouve ses vues dans un schéma `staging_staging`. Dites la cause et la correction.

E4. La jointure commode. Un collègue propose d'ajouter le nom du concessionnaire à `stg_commandes`, par une jointure vers `stg_concessionnaires` : « ça évitera de la refaire partout ensuite ». Répondez-lui, en donnant le principe et ce qui casse concrètement plus tard.

E5. Le chemin qui bouge. Un collègue lance `dbt run` depuis son dossier de projet, tout va bien. Le lendemain, il le lance depuis le dossier parent avec `--project-dir`, et reçoit une erreur de fichier introuvable sur la base. Dites la cause et la correction.

Questions à développer

A. Ce que `dbt` ne fait pas. On présente souvent `dbt` comme l'outil qui « transforme les données ». Expliquez en quoi cette formule est trompeuse, ce que `dbt` fait réellement, et ce que cela implique quand un modèle est lent.

B. Le staging qui ne joint pas. La règle paraît arbitraire, et elle coûte du travail : sans jointure, chaque consommateur du staging doit refaire les siennes. Défendez la règle, puis donnez le cas limite où vous accepteriez de la contourner et à quelles conditions.

C. (avancé) Déclarer ou pas. Ce cours déclare trois sources sur les vingt-deux tables de la RAW, au motif qu'on ne déclare que ce qu'on lit. Discutez ce choix : ce qu'il protège, ce qu'il coûte au moment où le projet grandit, et ce que vous feriez dans une équipe de cinq personnes.

Mises en situation

M1. Fromagerie de Saint-Prime. La fromagerie reçoit chaque semaine, de son distributeur, un fichier des ventes en épicerie : point de vente, produit, semaine, quantité. Le distributeur renvoie le fichier complet des douze dernières semaines à chaque livraison, si bien que la même semaine arrive douze fois, parfois avec des chiffres révisés. On vous demande de monter le staging. Dites ce que vous écrivez, quelle colonne fait office de lot, et le contrôle que vous posez avant de livrer quoi que ce soit à l'aval.

M2. (avancé) Scierie de Notre-Dame-du-Lac. Vous reprenez un projet dbt existant, quarante modèles. Vous découvrez que la moitié nomme les tables sources en dur, sans `source()`, et qu'un tiers des modèles se réfèrent entre eux sans `ref()`. Le projet tourne pourtant tous les matins. Décrivez ce que vous corrigez en premier, comment vous vous y prenez sans tout casser, et ce que vous pouvez promettre à votre chef pendant la reprise.

M3. Le staging qui ralentit. Chez un client, le staging est en vues, comme ici, mais une de ces vues est lue par une trentaine de modèles en aval, et chaque lecture relit huit millions de lignes brutes. L'exécution quotidienne passe de dix minutes à une heure. Dites ce que vous changez, ce que cela coûte, et le critère que vous retiendrez désormais pour choisir une matérialisation.